



WeCloudData

Agent Failure Modes

Introduction





WeCloudData

Common Agent Failure Patterns

Agent Failure Modes

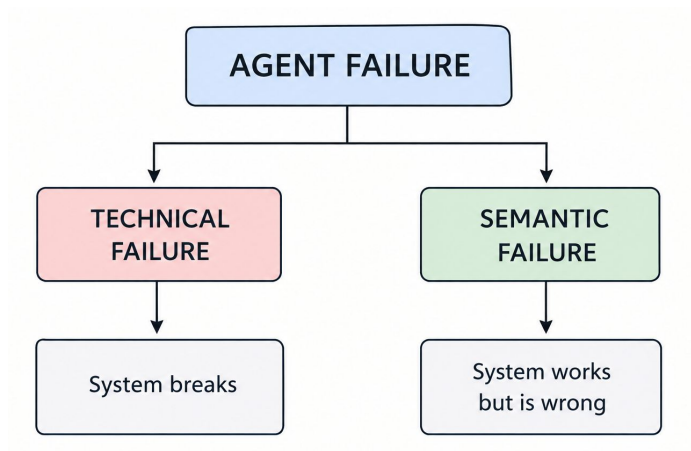


What Can Go Wrong?

Common Agent Failure Patterns

An agent can fail in two different ways:

Example:



Technical

- API timeout
- Tool unavailable
- Network failure

Semantic

- Wrong decision
- Wrong tool
- Wrong assumption
- False completion





Technical vs Semantic Failure

Common Agent Failure Patterns

Technical Failure	Semantic Failure
System breaks	System may appear successful
Usually produces an error	May produce no error
Easier to detect	Harder to detect
Infrastructure problem	Decision/behavior problem

A system can succeed technically but still fail semantically.



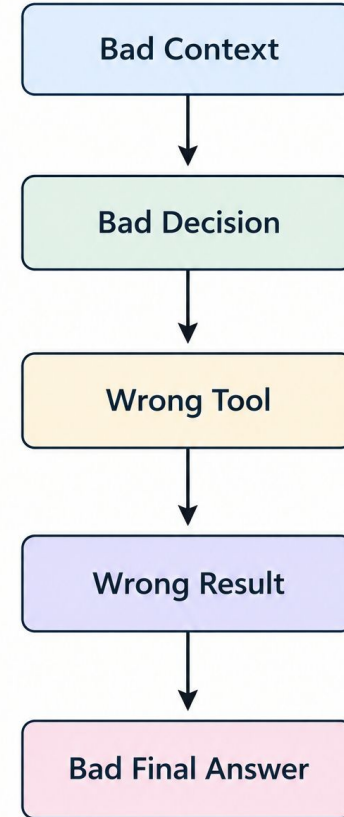
Failure Is Often a Chain

Common Agent Failure Patterns

A failure can spread through multiple steps.

Cascade effect:

An early error can propagate into downstream steps.





Why Failure Modes Matter?

Common Agent Failure Patterns

Benefits for Agent Developers:

Understanding failure modes helps us:

- Build more reliable agents
- Improve debugging workflows
- Create better evaluation systems
- Design stronger guardrails
- Reduce costly mistakes

Before Debugging

Ask:

1. Was the failure technical or semantic?
2. Which module introduced the error?
3. Did the error cascade through the workflow?
4. Is this a recurring pattern?

Agent reliability is not just about making better decisions, it is about understanding how bad decisions emerge and spread through the system.



Failure Taxonomy

Common Agent Failure Patterns

Failures can also be grouped by where they originate:

System / Specification

- Ambiguous requirements
- Poor system design

Inter-Agent Misalignment

- Communication problems
- Collaboration failures

Verification / Termination

- Premature stopping
- Missing quality checks

Use taxonomies to organize failures, not as universal rules.



WeCloudData

Agent System Breakdown

Agent Failure Modes



One Agent System Example

Agent System Breakdown

User asks:

"Research the latest developments in fusion energy and prepare a concise report with reliable sources."

Now let's break this system.





What Does Success Look Like?

Agent System Breakdown

Before looking for failures, define success.

The agent should:

- Research the requested topic
- Use appropriate sources
- Produce relevant findings
- Write a concise report
- Include reliable sources
- Complete the task only when requirements are met

You cannot detect failure without knowing what success means.

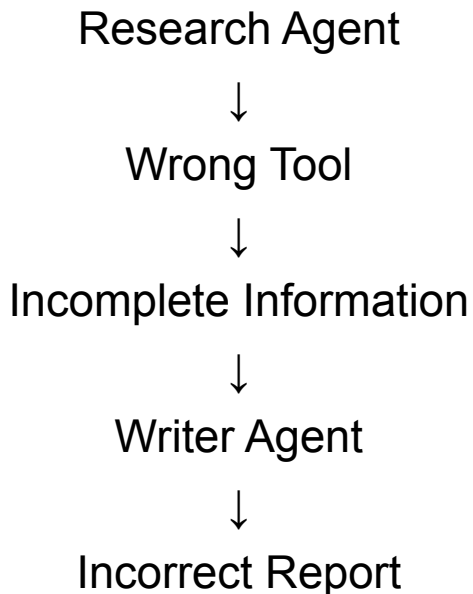




Failure 1: Wrong Tool

Agent System Breakdown

The Research Agent chooses an inappropriate tool.



Possible Safeguard?

- Clear tool descriptions
- Tool validation
- Appropriate permissions
- Result validation





Failure 2: Wrong Parameters

Agent System Breakdown

The agent selects the correct tool but sends incorrect information.

User Request



Parameter Extraction



Wrong Parameter



Tool Executes



Wrong Result

Example

User says: **"Meeting tomorrow at 3 PM."**

Agent extracts the wrong date.

Successful tool execution does not
guarantee a correct outcome.





Failure 3: Ghost Action

Agent System Breakdown

The agent claims that an action succeeded without verifying the external system.

Agent Response:

"Done, the report has been saved."

VS.

System State:

File was never saved.

Ghost Action

The agent claims an action happened, but the action was never executed.

The response says "done." The system says "not done."





False Task Completion

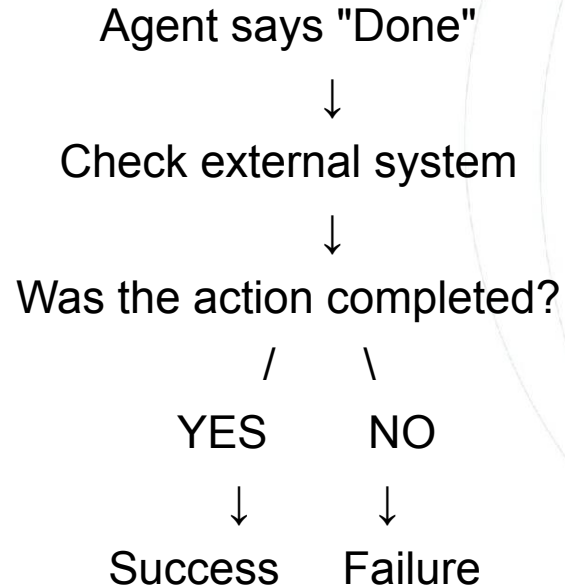
Agent System Breakdown

An agent can appear successful because:

- The response is fluent
- The response is relevant
- No technical error occurred
- The user receives confirmation

Never treat a confident response as proof of completion.

But

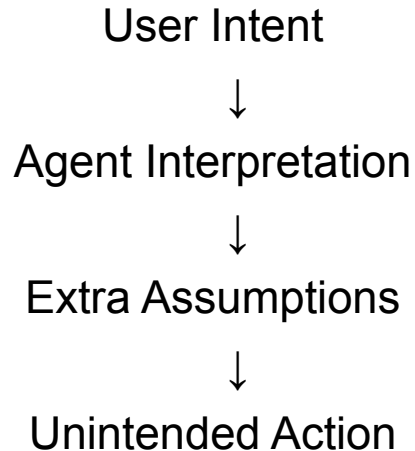




Failure 4: Semantic Over-Interpretation

Agent System Breakdown

The agent understands the request too broadly.



Example:

User asks: "Find flights to Dubai."

Agent assumes:

- Specific travel date
- Preferred airline
- Budget
- Seat type

Problem: The agent acted on information the user never provided.



Failure 5: Too Much Access

Agent System Breakdown

Giving an agent more access than necessary increases risk.

Too Much Access



More Possible Actions



More Possible Damage

Research Agent

- ✓ Read web sources
- ✓ Search documents
- ✗ Delete files
- ✗ Modify production data

Least Privilege

Give the agent only the permissions required for the task.



WeCloudData

Loops and Termination

Agent Failure Modes



When an Agent Gets Stuck

Loops and Termination

An agent can repeatedly perform the same action without making progress.

Search



Search again



Search again



Search again



...

Common symptoms:

- Same tool call repeated
- Similar outputs repeated
- Same question repeated
- Same information reconsidered
- No completion





Activity $\neq$ Progress

Loops and Termination

Example:

Step 1: Analyze document
Step 2: Improve summary
Step 3: Improve summary again
Step 4: Improve summary again
Step 5: Improve summary again...

The agent is active and busy, but no real progress is being made.

Activity $\neq$ Progress

An agent can appear productive while being trapped in a repetitive cycle.

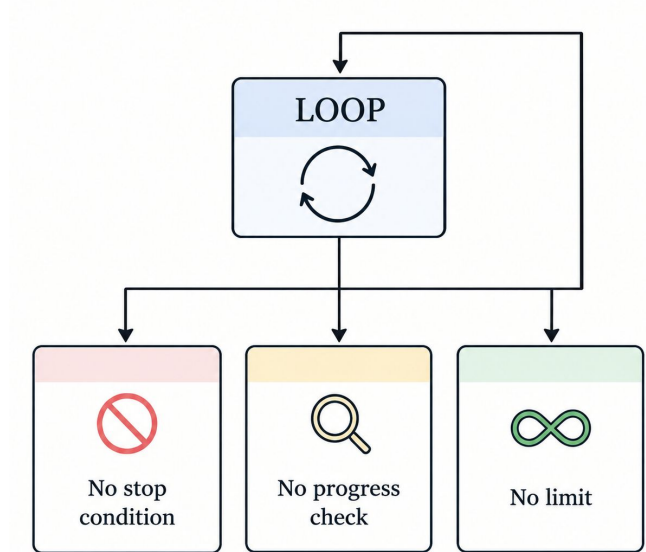


Why Does the Loop Continue?

Loops and Termination

A loop can happen when the system has:

- No completion condition
- No progress check
- No retry limit
- No meaningful state change





Refinement Loops

Loops and Termination

A refinement loop happens when the agent keeps improving instead of finishing.

Generate



Improve



Improve



...

Problem: The agent never decides that the output is good enough.

Safeguards:

- Completion criteria
- Maximum iterations
- Progress checks
- Quality threshold

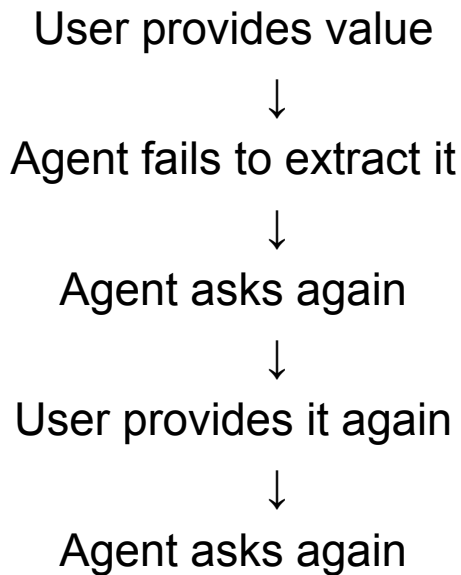




Interrogation Loop

Loops and Termination

An agent repeatedly asks for information that the user already provided.



Possible root cause:

Silent parameter extraction failure.

Impact:

- User frustration
- Lower trust
- Poor conversation quality



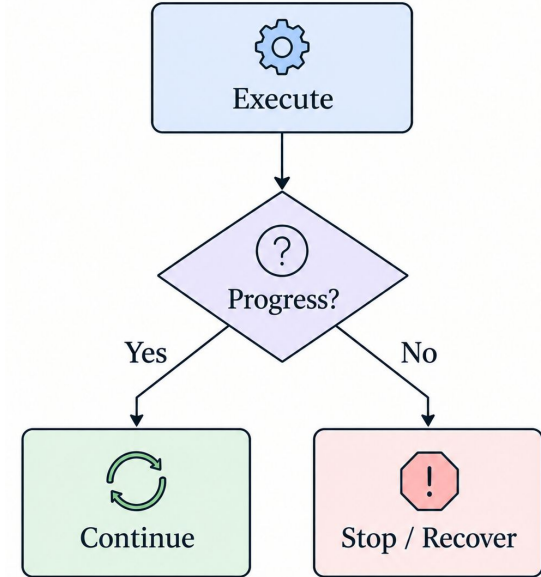


Stopping Safely

Loops and Termination

An agent should stop when:

- The goal is completed
- Completion is verified
- A retry limit is reached
- No meaningful progress is occurring
- A safety rule is triggered
- Execution must pause for human approval



A reliable agent knows when to stop.



WeCloudData

Observability and Diagnosis

Agent Failure Modes



How Do We Find the Failure?

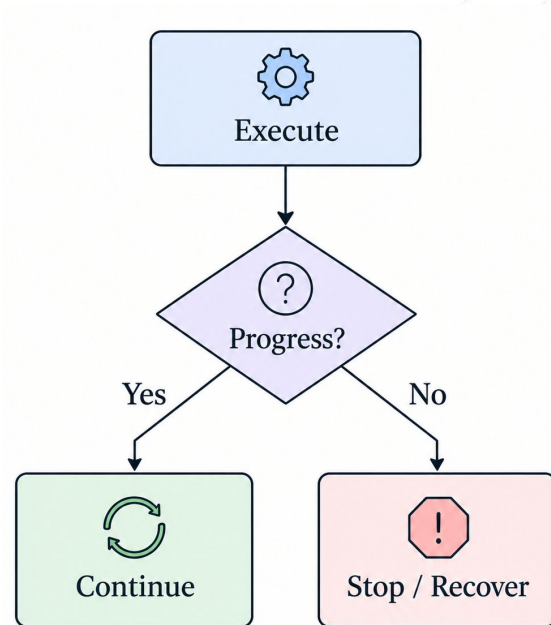
Observability and Diagnosis

The question is no longer:

"Did the system crash?"

Instead:

"What happened during execution?"



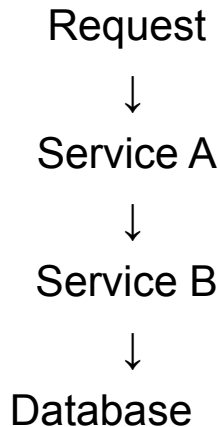
We need evidence from the execution path.



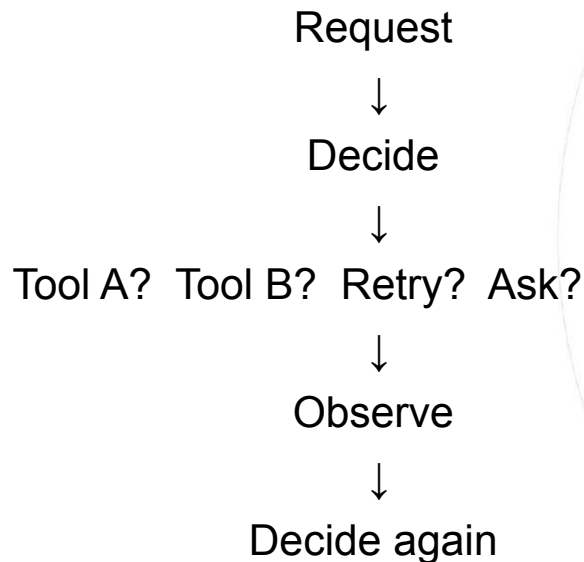
Why Traditional Logs Are Not Enough?

Observability and Diagnosis

Traditional systems often follow predictable paths:



Agent systems can behave dynamically:



Agent systems require visibility into what happened during execution, not just whether the application crashed.

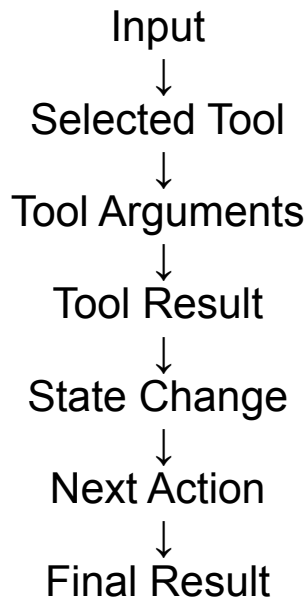




What Should We Observe?

Observability and Diagnosis

Capture useful execution evidence:



Also useful:

- Errors
- Retries
- Guardrail decisions
- Latency
- Token/cost information

Observability is about understanding execution, not exposing private chain-of-thought.





Traces vs. Spans

Observability and Diagnosis

Traces	Spans
The complete end-to-end workflow.	A single step within the workflow.
Examples: ● Memory retrieval ● Tool call ● Planning step ● Database query	Trace — Span 1: Memory Retrieval — Span 2: Planning — Span 3: Tool Selection — Span 4: API Call — Span 5: Final Response

Trace = whole story

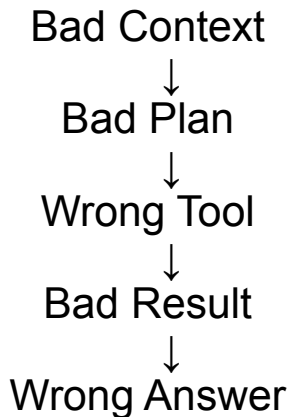
Span = one part of the story



Find the First Failure

Observability and Diagnosis

A final bad answer may have many earlier causes.



Ask:

**Where did the failure
actually begin?**

Fix the **first meaningful failure**, not just
the final symptom.





Failure → Detection → Safeguard

Observability and Diagnosis

Use this structure to analyze each failure.

Failure	Detection	Safeguard
Wrong tool	Check tool choice	Tool validation
Wrong parameters	Validate arguments	Schema/rules
Ghost action	Check external state	Action verification
Loop	Detect repeated steps	Retry/iteration limit
Wrong assumption	Validate requirements	Confirmation
Too much access	Audit permissions	Least privilege

Reliable systems do not only detect failures, they control them.



WeCloudData

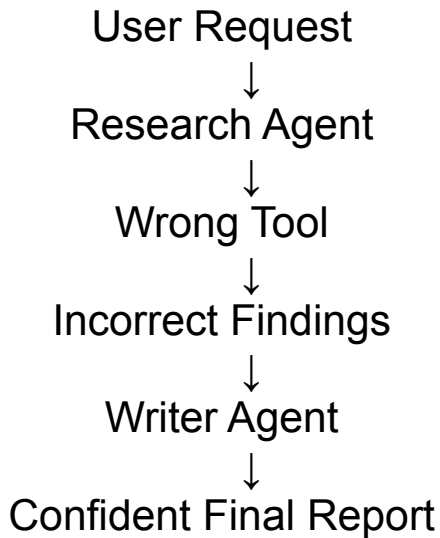
Putting it all Together Agent Failure Modes



Complete Failure Story

Putting it all Together

Let's break the Research → Writer system:



What happened?

- Tool selection was wrong
- Tool technically executed
- Result looked valid
- Writer trusted the result
- Error propagated to the final answer

This is a semantic + cascading failure.





Diagnose the Execution Trace

Putting it all Together

Let's break the Research → Writer system:

Trace

✓ User request received
✓ Research task created ✗ Wrong tool selected ← FIRST FAILURE
✓ Tool executed
✓ Result returned
✓ Writer generated report
✓ Final response delivered

Diagnosis

Failure: Semantic
Origin: Tool selection
Cascade: Yes
Final symptom: Incorrect report

Fix

Tool Selection
↓
Validate
↓
Execute
↓
Validate Result
↓
Continue





Summary

- Agents can fail technically or semantically.
- A system can work technically but still produce the wrong result.
- Early mistakes can spread through the workflow.
- Tool calls can be successful but still be incorrect.
- Agents can claim actions that never happened.
- Loops can keep an agent busy without making progress.
- Completion should be verified before declaring success.
- Execution traces help us understand what happened.
- Find the **first meaningful failure**, not just the final symptom.
- Reliable agents need **validation, boundaries, stopping conditions, and safeguards**.



WeCloudData

Thank You

We'd love to hear from you. If you have any questions,
feel free to reach out to our team anytime!